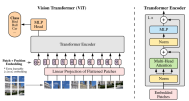
Advanced Artificial Intelligence and Machine Learning

Transformers Outside NLP



Learning goals

- Computer vision: what it is, its tasks and applications
- From the NLP Transformer to the Vision Transformer: patches as tokens, inductive bias, scale
- Transformers for detection and segmentation: Swin, DETR, MaskFormer
- Multimodality: vision language models (CLIP, BLIP) and multipurpose models (Gato, OFA)
- Video: from frames to video Transformers (ViViT, TimeSformer)

What Is Computer Vision

DEFINITION

- The extraction of meaningful information from an image input is at the core of computer vision.
- **Computer vision** is the science and technology of making machines see.
- It involves the development of theoretical and algorithmic methods to acquire, process, analyze, and understand visual data, and to use this information to produce meaningful representations, descriptions, and interpretations of the world (Forsyth & Ponce, *Computer Vision: A Modern Approach*).

DEEP LEARNING AND THE COMPUTER VISION RENAISSANCE

- The seeds were sown in the 1960s: pioneers like David Marr and Hans Moravec asked whether machines can be made to see; early breakthroughs such as edge detection and object recognition came from a mix of cleverness and brute force.
- Classical pipeline: extract features with image-preprocessing techniques, then apply a classical machine learning algorithm to the features — simpler than hard-coded rules, but still dependent on domain knowledge and exhaustive feature engineering.
- When deep learning methods and large datasets meet, machines **learn complex features automatically from the raw data**.
- This paradigm shift allowed more adaptive and sophisticated models, causing a renaissance in the field.

IMAGE UNDERSTANDING: THREE LEVELS

- **Low-level processes**: primitive operations on images (e.g. sharpening, changing the contrast). Input and output are images.
- **Mid-level processes**: segmentation, description of objects, object classification. The input is an image, the result is attributes associated with the image.
- **High-level processes**: making sense of the entirety of an image, e.g. recognition of a given object, scene reconstruction, image-to-text. Tasks typically associated with human cognition.
- Image analysis is mainly concerned with low- and mid-level processes; computer vision with mid- and high-level processes.
- Sometimes the layers of a neural network perform such preprocessing automatically, eliminating the need for explicit preprocessing.

COMPUTER VISION TASKS OVERVIEW

Core tasks:

- Scene recognition
- Object recognition
- Object detection
- Segmentation (instance, semantic)
- Tracking
- Dynamic environment adaptation
- Path planning

Many more (non-exhaustive):

- Image captioning
- Image classification
- Image description
- Anomaly detection
- Image generation
- Image restoration
- Autonomous exploration
- Localization

TASK COMPLEXITY

- The complexity of a task is not determined by how difficult it seems to an informed audience, but primarily by the properties of the image or data being analyzed.
- Example: identifying a pedestrian looks simple to a human observer — computationally, its complexity varies with lighting conditions, occlusions, image resolution, and camera quality.
- In low light or with pixelated images, even basic pedestrian detection can become exceedingly complex, requiring advanced image enhancement and machine learning techniques.
- The challenge often lies in the intricacies of the visual data and the computational methods needed to extract meaningful insights from it.

Applications of Computer Vision

AUTONOMOUS VEHICLES, RETAIL AND E-COMMERCE

- **Autonomous vehicles**: cameras and sensors identify objects, pedestrians, traffic signs, lane markings, and other vehicles; computer vision algorithms help make real-time decisions such as steering, accelerating, or braking.
- **Retail and e-commerce**:
  - object recognition and recommendation: identify products in images or videos, understand their features, recommend similar products;
  - in physical stores: track inventory levels, detect stock shortages, analyze customer behavior to optimize store layouts.

QUALITY CONTROL IN ASSEMBLY LINES

- **Defect detection**: analyze products in real time and spot imperfections, scratches, dents, or incorrect assembly that might not be visible to the human eye, by comparing against a standard reference.
- **Automated inspection**: replaces time-consuming manual inspection that is prone to human error; cameras capture products moving along the line and machine learning detects deviations from the standard.
- **Real-time feedback and maintenance**: a detected defect can trigger immediate actions — alerting an operator, diverting the faulty product, or adjusting the machinery.

MEDICAL IMAGE ANALYSIS

- Interpret and extract information from X-rays, CT scans, MRIs, ultrasounds, and histopathology slides.
- **Diagnostic assistance**: detect abnormalities such as tumors and fractures; highlight areas of concern.
- **Segmentation and detection**: isolate organs, tissues, or pathologies, e.g. segment tumors in MRI or CT scans for treatment planning.
- **Treatment planning and monitoring**: precise measurements, tracking changes over time.
- **AI-assisted radiology**: automate routine tasks, flag potentially abnormal findings.
- **Drug development and research**: analyze cellular structures, tissue samples, or genetic material.

CHALLENGES FOR COMPUTER VISION SYSTEMS

- **Variability in data**: lighting, viewpoint, occlusions, backgrounds.
- **Scalability**: large datasets and real-time processing requirements.
- **Accuracy** in complex or cluttered scenes, with noise, irrelevant features, poor image quality.
- **Robustness to noise**: defects, sensor artifacts, distortions.
- **Integration with other technologies**: NLP, robotics, augmented reality.
- **Privacy and ethical concerns**: surveillance, facial recognition, data gathering.
- **Real-time processing**: e.g. autonomous vehicles, augmented reality.
- **Long-term reliability**, **generalization** across contexts and domains, **calibration and maintenance** of cameras and sensors.

Transformers in Computer Vision

TRANSFORMERS IN COMPUTER VISION

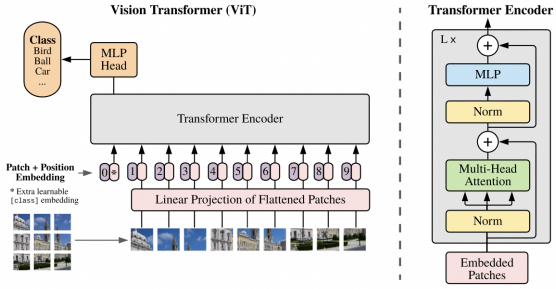
- Since the first appearance of the Transformer architecture in 2017, it has become an irreplaceable part of NLP models.
- Main advantage: Transformers can be trained on a large text corpus and then fine-tuned on a smaller task-specific dataset — enabling models with more than 100B parameters.
- Computer vision, however, still relied on convolutional architectures.
- With datasets constantly growing, researchers wanted to bring the Transformer architecture to computer vision:
  - some works combine CNN-like architectures with self-attention;
  - others replace convolutions entirely ▸ Ramachandran et al., 2019.
- Due to specialized attention patterns, these had not yet been scaled effectively on modern hardware accelerators — so in large-scale image recognition, classic ResNet-like architectures were still state-of-the-art.

AN IMAGE IS WORTH 16×16 WORDS

- The Google Brain team introduced a Transformer-based architecture for computer vision: the **Vision Transformer (ViT)** ▸ Dosovitskiy et al., 2020.
- Aim: apply a *standard* Transformer directly to images, with as few changes to the existing architecture as possible.
- The image is split into patches, and linear embeddings of these patches are the inputs to the Transformer.
- **These patches are the same as tokens (e.g. words) in NLP.**
- The model is trained for image classification in a supervised fashion.

VISION TRANSFORMER

- Goal: a simple but universally scalable architecture that follows the original Transformer.



▸ Source: Dosovitskiy et al., 2020

VIT METHOD: IMAGES AS SEQUENCES OF PATCHES

- In NLP, the Transformer input is a 1-dimensional sequence of token embeddings; images are 2-dimensional objects.
- To stay as close as possible to the original architecture, an image $x \in \mathbb{R}^{H \times W \times C}$ is reshaped into a sequence of flattened 2D patches $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$:
  - (H, W): resolution of the original image, C: number of channels,
  - (P, P): resolution of each patch (usually 16×16),
  - $N = HW/P^2$: number of patches — the Transformer's effective input sequence length.
- The Transformer uses a fixed vector size D through all layers: the flattened patches are mapped to D dimensions with a trainable linear projection $\Rightarrow$ **patch embeddings**.

VIT METHOD: CLASS TOKEN AND POSITION EMBEDDINGS

$$z_0 = \left[x_{\text{class}}; x_p^1 E; x_p^2 E; \cdots; x_p^N E\right] + E_{pos}, \qquad E \in \mathbb{R}^{(P^2 \cdot C) \times D},\ E_{pos} \in \mathbb{R}^{(N+1) \times D}$$

- A learnable `[class]` token is prepended to the patch embeddings — **like in BERT**.
- Its state at the output of the encoder, after layer norm, is the image representation:
$$y = \text{LN}\left(z_L^0\right)$$
- The classification head is attached only to this token: an MLP with one hidden layer during pre-training, a single linear layer at fine-tuning time.
- Standard learnable 1-dimensional **position embeddings** are added to the patch embeddings.

VIT METHOD: THE ENCODER

- The standard Transformer encoder: alternating layers of multi-headed self-attention (MSA) and MLP blocks, layer norm (LN) before each block, a residual connection after each block:

$$\mathbf{z}'_\ell = \mathrm{MSA}\left(\mathrm{LN}\left(\mathbf{z}_{\ell-1}\right)\right) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L$$

$$\mathbf{z}_\ell = \mathrm{MLP}\left(\mathrm{LN}\left(\mathbf{z}'_\ell\right)\right) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L$$

Newer ViTs update this block as discussed in session 9, e.g. SwiGLU and RoPE ▸ EVA-02, QK-norm and parallel layers ▸ ViT-22B.

CNNS VS. VISION TRANSFORMERS: INDUCTIVE BIAS

- **Inductive bias**: the set of assumptions a learning algorithm uses to make predictions — a shortcut that helps a model make educated guesses based on what it has seen so far.
- Two inductive biases of CNNs:
  - **Translational equivariance**: an object can appear anywhere in the image, and CNNs can detect its features.
  - **Locality**: pixels interact mainly with their surrounding pixels to form features.
- ViT does not have these assumptions. That is why, up to a certain dataset size, CNNs are actually better than ViT.
- But: being (mostly) different types of linear functions, the Transformer makes ViT highly scalable — which lets ViT overcome the missing inductive biases with massive amounts of data.

VIT METHOD: INDUCTIVE BIAS

- ViT has a significantly lower image-specific inductive bias than CNNs:
  - only the MLP layers are local and translationally equivariant; the self-attention layers are global;
  - the 2-dimensional neighborhood structure is used sparingly: the image is cut into patches at the beginning, and position embeddings are resized as needed at fine-tuning time.
- Resolution adjustment and patch extraction are the *only* points where an inductive bias about the 2D structure of images is injected manually.
- Alternative (hybrid): the input sequence can be a CNN's feature map, on which the patch embedding projection is applied.

VIT: PRE-TRAINING AND FINE-TUNING

- Vision Transformers are pre-trained on large datasets and fine-tuned to (smaller) downstream tasks.
- For fine-tuning, the pre-training projection head is removed and a zero-initialized $D \times K$ feed-forward layer is attached, $K =$ number of downstream classes.
- It is beneficial to fine-tune at a higher resolution than in pre-training; ViT can handle arbitrary sequence lengths.

VIT: EXPERIMENTS

- Similarly to BERT, multiple versions at various scales: Base (B), Large (L), Huge (H) with 12, 24 and 32 layers and 86M, 307M and 632M parameters.
- Compared on ImageNet against a slightly modified ResNet, "ResNet (BiT)" (group normalization, standardized convolutions), and Noisy Student, a large EfficientNet ▸ Xie et al., 2019:
  - ViT-Huge with 14×14 patches: 88.5 % accuracy,
  - ResNet (BiT): 87.54 %, Noisy Student: 88.4 %,
  - ViT-Large with 16×16 patches: 87.76 %.
- ViT outperforms CNN-based architectures on all larger datasets, yet performs slightly worse than CNNs on smaller datasets.

Vision Transformers for Image Classification

TRANSFER LEARNING & FINE-TUNING

- Training a ViT on millions of images is not feasible for everyone. Instead, use openly available pre-trained weights (e.g. the Hugging Face Hub).
- **Transfer learning**: leverage the features a ViT learned on a very large dataset and apply them to our (possibly small) dataset.
  - By freezing most of the weights, we train only certain layers: good performance with less training time and low GPU consumption.
- **Fine-tuning**: if our domain differs from the pre-training data, update the weights of the entire pre-trained model, with a lower learning rate.
- In most scenarios, transfer learning is ample for Vision Transformers.
- Both work for **multi-class** (one label per image) and **multi-label** (several labels per image) classification.

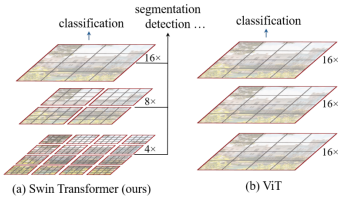
Swin Transformer

SWIN TRANSFORMER

- Introduced in 2021 Liu et al., 2021: optimizes for latency and performance using a **shifted window** (as opposed to sliding window) approach, which reduces the number of operations required.
- Swin is a **hierarchical backbone** for computer vision, e.g. for image classification.
  - A *backbone* is the part of a network that does feature extraction; additional layers can be added on top for a variety of vision tasks.
  - *Hierarchical* backbones have tiered structures, sometimes with varying resolutions — in contrast to the non-hierarchical *plain* backbone of e.g. ViTDet.

SHIFTED WINDOWS

- In the original ViT, attention is done between each patch and all other patches — computationally intensive.
- Swin reduces ViT's quadratic complexity to **linear** complexity (w.r.t. image size): similar to a CNN, patches only attend to other patches in the **same window**, and are then gradually merged with neighboring patches. This is what makes Swin hierarchical.



▸ Source: Liu et al., 2021

SWIN: ADVANTAGES AND SUCCESSORS

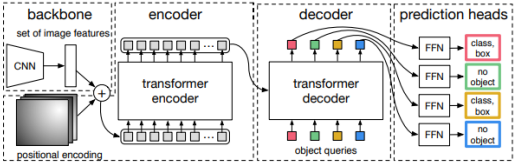
- **Computational efficiency**: Swin is more performant than completely patch-based approaches like ViT.
- **Large datasets**: as training size goes up, Swin outperforms CNNs; the large number of parameters enables increased capacity for learning more complex representations.
- **Swin Transformer V2** Liu et al., 2022: one of the first 3B-parameter vision models, trainable with high-resolution images. It stabilizes training, transfers models pre-trained on low-resolution images to high-resolution tasks, and uses SimMIM, a self-supervised approach that reduces the number of labeled images required.
- **Image restoration**: SwinIR turns low-resolution into high-resolution images; Swin2SR improves on it using Swin V2.

DEtection TRansformer (DETR)

OBJECT DETECTION WITH DETR

- **Object detection**: detect objects in an image. Input: e.g. an image of a road. Output: e.g. `[('car',X1,Y1,W1,H1), ('pedestrian',X2,Y2,W2,H2)]` — class, plus the x, y coordinates, width and height of each bounding box.
- Traditional detectors like YOLO use hand-crafted features such as **anchor box priors** (initial guesses of object locations and shapes), and post-processing steps to remove overlapping boxes, which need carefully selected filtering heuristics.
- **DETR** ▸ Carion et al., 2020 simplifies the detector: an encoder-decoder Transformer after the feature-extraction backbone **directly predicts bounding boxes in parallel**, requiring minimal post-processing.

DETR: ARCHITECTURE

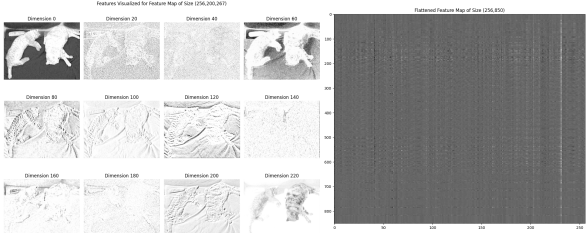


Source: Carion et al., 2020

- A CNN backbone; its output is fed into a Transformer encoder, resulting in N embeddings.
- The decoder uses learned positional embeddings (**object queries**) and the encoder output, generating another N embeddings.
- Each of the N embeddings goes through feed-forward layers predicting the bounding box (coordinates, width, height) and the object class — or that there is no object.

DETR: ENCODER-DECODER

- The encoder expects a sequence, so the backbone's feature map of size `[dimension, height, width]` is downsized and flattened to `[dimension, less than height` $\times$ `width]`.



Left: 12 of 256 feature-map dimensions; each extracts some features of the cat image (some focus on the cat's patterns, some on the bed sheets). Right: keeping 256 dimensions, width and height are flattened into size 850.

DETR: POSITIONAL EMBEDDINGS

- Transformers are permutation invariant, so positional embeddings are added in both encoder and decoder to remind the model where in the image an embedding comes from.
- **Encoder: fixed** positional encodings, as in the original Transformer — sinusoidal functions of varying frequencies at different feature dimensions. A sense of position without learned parameters.
- **Decoder: learned** positional encodings (the object queries) — each position has its own encoding, learned during training.

DETR: SET-BASED GLOBAL LOSS

- YOLO's loss: bounding box, objectness (probability that an object exists in a region of interest) and class loss, computed over a fixed number of boxes per grid cell.
- DETR generates unique bounding boxes in a **permutation-invariant** manner: the order of the detections does not matter, and the boxes must differ from each other. So predictions must be **matched** to the ground truth before they can be scored.
- **Bipartite matching**: a one-to-one matching between ground-truth and predicted boxes, pairing those with the highest similarity in box and class.
  - The closest prediction is matched with the corresponding ground truth, so boxes and classes are adjusted properly in the loss.
  - Without matching, a correct prediction that is not in the same order as the ground truth would be marked incorrect.

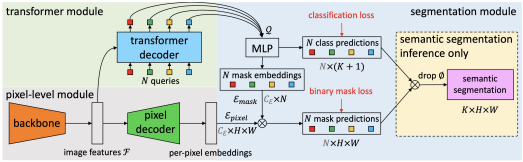
Transformer-Based Image Segmentation

CNNS VS. TRANSFORMERS FOR SEGMENTATION

Before ViTs, CNNs (U-Net, Mask R-CNN) were the go-to choice for segmentation. Limitations Transformers aim to solve:

- **Spatial limitations**: CNNs learn local patterns through small receptive fields, so it is hard to link features that are far apart but related. ViTs capture global dependencies via attention, considering the entire image at once.
- **Task-specific components**: Mask R-CNN uses hand-designed components (e.g. spatial anchors) that add complexity and require manual tuning. ViT-based methods eliminate them.
- **Task specialization**: CNN-based models treat semantic, instance and panoptic segmentation separately. ViT-based models such as MaskFormer, SegFormer or SAM handle all three in a single framework.

MASKFORMER



▸ Source: Cheng et al., 2021

- **Pixel-level module**: a backbone extracts image features; a pixel decoder generates per-pixel embeddings.
- **Transformer module**: a standard Transformer decoder computes per-segment embeddings from image features and learnable queries.
- **Segmentation module**: class predictions (linear classifier) and mask embeddings (MLP) per segment; mask embeddings combined with per-pixel embeddings give a binary mask per segment.

MASKFORMER: TRAINING AND FINE-TUNING

- MaskFormer predicts segmentation masks for each class present in an image, unifying semantic and instance segmentation in one architecture.
- Trained with a binary mask loss — the same one as DETR — plus a cross-entropy classification loss per predicted segment.
- **Panoptic segmentation**: label every pixel with its category *and* identify distinct objects within those categories (semantic + instance). MaskFormer's output contains multiple instances of the same class, each with its own mask.
- Transformer-based segmentation models are data-hungry and hard to train from scratch, so transfer learning is common: typically the backbone, the pixel decoder and the Transformer decoder are frozen, and the rest is fine-tuned for new class predictions and masks.

Exploring Multimodal Text and Vision Models

FROM VISION TO MULTIMODALITY

- The Transformer architecture revolutionized NLP, but did not stop at the text modality.
- It has begun to conquer the field of vision (including image and video), bringing with it a wide array of new research and applications.
- Next: the **data fusion** possibilities that the modality-overlapping use of Transformers has enabled, and the tasks and models that benefit:
  - why blending text and images is crucial;
  - multimodal tasks such as image-to-text, text-to-image, visual question answering (VQA), captioning, visual reasoning;
  - vision language models such as CLIP and its relatives.

A Multimodal World

WHAT IS MULTIMODALITY?

- A **modality** is a medium or a way in which something exists or is done.
- Humans use 5 sense organs; each is a different modality through which information comes to us — hence *multimodality*.
- Example: on a windy night you hear an eerie sound. Relying on your ears alone, you cannot tell its source; once you turn on the lights and look around, you see a half-open window.
- Adding modalities to the same situation lets them assist each other and improves the information content.

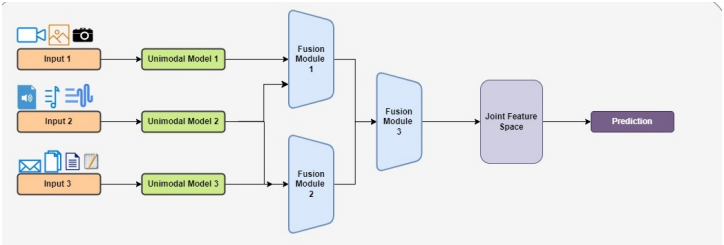
MODALITIES IN DEEP LEARNING

- In deep learning, each modality is a way data arrives to a model. Most common: **vision, audio, text**; others for specific use cases: LIDAR, EEG data, eye-tracking data, ...
- Unimodal models and datasets have been studied for long, with many tasks and benchmarks, but are limited: a single modality might not give the complete picture.
- Combining modalities increases the information content and reduces the chance of missing cues.
- Common combinations:
  - vision + text: infographics, memes, articles, blogs;
  - vision + audio: a video call, dyadic conversations;
  - vision + audio + text: videos or movies with captions, social media content;
  - audio + text: voice notes, music files with lyrics.

MULTIMODAL DATASETS AND TASKS

- A dataset with multiple modalities is a **multimodal dataset**, e.g.
  - vision + text: Visual Storytelling, Visual Question Answering, LAION-5B;
  - vision + audio: VGG-Sound, RAVDESS; vision + audio + text: RECOLA, IEMOCAP.
- A task involving two or more modalities is a **multimodal task**; typically, two different modalities at the input and output ends:
  - vision + text: visual question answering, image-to-text (captioning, OCR), text-to-image (generation), text-to-video;
  - audio + text: automatic speech recognition, text-to-speech.
- Example: multimodal emotion recognition from audio+text, text+vision, audio+vision or all three is more efficient than unimodal emotion recognition.

MULTIMODAL MODELS: FUSION



- A **multimodal model** processes data from multiple modalities at the same time.
- Unimodal models encode each input; a **fusion module** combines the encoded data (early, late, or hybrid fusion) into a joint representation; a classification network makes predictions from it.

APPLICATION: MULTIMODAL SEARCH

- Online content is largely multimodal: an image comes with text that describes it; many companies hold large, multimodal, mostly unstructured databases.
- **Vision language models (VLMs)** understand and process both vision and text and can serve as candidates for multimodal search.
- Idea: map text and image pairs to a **joint embedding space** where image and text embeddings that are similar in meaning lie close together.
- This enables text-to-image search and vice versa, and other downstream tasks on the same embeddings.

Multimodal Architectures

MULTIMODAL ARCHITECTURES: OVERVIEW

- **Multimodal learning**: learning representations from different types of input modalities, such as image data, text or speech.
- Methodological breakthroughs in NLP and computer vision made multimodal models attractive: they strengthen predictions and better emulate the way humans learn.
- With images and text as input data, five families:
  1. **Image2Text**: e.g. captioning
  2. **Text2Image**: generating images from text
  3. **Images supporting language models**
  4. **Text supporting vision models**
  5. **Models for both modalities**

IMAGE2TEXT

- **Image2Text**: describing a photograph or parsing a complex scene is easy for humans, much harder for computers.
  - MS COCO (*Common Objects in Context*, 2014) addresses three core problems in scene understanding: object detection, segmentation, and captioning.
  - Transformer-based architectures, e.g. the Meshed-Memory Transformer (M^2), improve both image encoding and language generation for captioning.

TEXT2IMAGE

- **Text2Image**: incorporating textual input to generate visual representations.
  - Made possible largely by NLP breakthroughs in contextual text representations: Transformer-like architectures encode the input into embedding vectors that guide image generation.
  - Roots: GANs, VAEs, VQ-VAE, diffusion, and autoregressive models.

IMAGES SUPPORTING LANGUAGE MODELS

- Distributional models such as word2vec and BERT assume that meaning comes from the context in which words appear in a text corpus.
- Historically questioned: words and sentences must be **grounded** in other perceptual dimensions to understand their meaning.
- Starting point: a language model into which visual representations are integrated. A spectrum of solutions:
  - textual and visual elements learned separately, then combined;
  - textual and visual features learned jointly.
- Often better than pure language models, but they still struggle, e.g. with abstract words and sentences.

TEXT SUPPORTING VISION MODELS

- Natural language as additional supervision for computer vision models: much more signal than in manually labeled data alone.
- Prominent example: **CLIP** (*Contrastive Language-Image Pre-training*), with the WIT dataset of 400 million text-image pairs scraped from the internet.
  - A text encoder and an image encoder are pre-trained jointly with a contrastive goal: predict which text belongs to which image in a batch — more efficient than generating captions.
  - At test time, the learned text encoder works as a "zero-shot" classifier on embeddings of the target dataset's classes.
- Inspired by NLP's pre-training directly on raw text (e.g. GPT-n).

MODELS FOR BOTH MODALITIES

- Goal: incorporate text and image inputs into a single unifying framework, closer to general self-supervised learning.
- Advantages: no human labelling, so no capacity constraints of supervised models; rather than potentially learning modality-specific biases, a general multipurpose framework can help increase robustness while simplifying the learner portfolio.

MODELS FOR BOTH MODALITIES: EXAMPLES

- Examples:
  - **data2vec**: one self-supervised framework for speech, language, or vision — predict latent representations of the full input from a masked view, with a standard Transformer;
  - **ViLBERT**: an earlier model that handles cross-modality tasks;
  - **Flamingo**: a few-shot model with 80B parameters, using a large language model for open-ended text generation.

Introduction to Vision Language Models

OUR WORLD IS MULTIMODAL

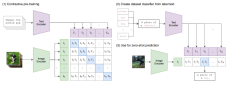
- Humans explore the world through diverse senses: sight, sound, touch, scent. Machine learning seeks models that process and integrate data across multiple **modalities**.
- Two key challenges:
  - **Representation**: effectively summarizing multimodal data, capturing the connections among individual modality elements.
  - **Alignment**: identifying connections and interactions across all elements.
- Inherent difficulties: one modality may dominate the others; additional modalities can introduce noise; full coverage over all modalities is not guaranteed; modalities can have complicated relationships.
- This chapter: the fusion of vision and language — **Vision Language Models (VLMs)**.

FROM TRAINING FROM SCRATCH TO PRE-TRAINING

- Generating text from images (captioning, visual question answering) has been studied for years.
- A shift from training from scratch to the paradigm first proposed in ULMFiT ▸ Howard and Ruder, 2018:
  - pre-train the model with extensive training data,
  - fine-tune it with task-specific data,
  - use it for downstream tasks such as classification.
- VLMs extend this paradigm to images. In 2021, OpenAI's **CLIP** used an image-text **contrastive** objective: paired images and texts are brought close in the embedding space, all others pushed apart.
- This captures rich vision-language correspondence, enabling **zero-shot** predictions by matching embeddings of any image and text. CLIP outperformed ImageNet models on out-of-distribution tasks.

VLM MECHANISM

- Given image-text pairs: extract features with a text encoder and an image encoder (CNN- or Transformer-based), then learn the vision-language correlation with a pre-training objective:
  - **contrastive**: pull paired samples close, push others far away in the embedding space;
  - **generative**: learn semantic features by generating image/text data;
  - **alignment**: global image-text matching or local region-word matching.
- With the learned correlation, VLMs are evaluated zero-shot by matching embeddings of any given images and texts.



▸ Source: Radford et al., 2021

VLM STRATEGIES

Grouping VLMs by how they combine the two modes of learning:

- **Images as tokens**: divide images into patches, treat each patch as a "token", and train on joint sequences of text and image tokens (VisualBERT, SimVLM).
- **Image embeddings as a prefix** for a *frozen* pre-trained language model: learn an image embedding space compatible with the language model's, without changing its parameters (Frozen, ClipCap).
- **Cross-attention**: a specially designed cross-attention mechanism fuses visual information into the layers of the language model (VisualGPT).
- **No training**: combine vision and language models via guided decoding of the next token (MAGiC).

Research focus: collecting large-scale informative image-text data, designing effective models for big data, and new pre-training objectives.

COMMON VLM DATASETS

- **MSCOCO**: 328K images, each paired with 5 independent captions.
- **NoCaps**: measures generalization to unseen classes and concepts (in-domain, near-domain, out-of-domain).
- **Conceptual Captions**: 3 million image-caption pairs, mined from the web and post-processed.
- **ALIGN**: a noisy dataset of over one billion image alt-text pairs, without expensive filtering or post-processing.
- **LAION**: image-text pairs extracted from Common Crawl (web pages crawled 2014–2021); used to train Stable Diffusion.

DOWNSTREAM TASKS AND EVALUATION

- Downstream tasks: image classification, object detection, semantic segmentation, image-text retrieval, action recognition.
- **Zero-shot prediction** (most common): apply the pre-trained VLM directly, without task-specific fine-tuning.
- **Linear probing**: freeze the VLM and train a linear classifier on its embeddings, measuring the quality of the representation.
- Harder datasets probe generalization: **Hateful Memes** adds difficult "benign confounders"; **Winoground** tests whether models truly grasp compositional relationships.



(a) some plants surrounding a lightbulb

(b) a lightbulb surrounding some plants

Figure 1. An example from Winoground. The two sentences contain the same words but in a different order. The task of understanding which image and caption match is trivial for humans but much harder for vision and language models. Every model that we tested (UNITER, ViLLA, VinVL, VisualBERT, ViLT, LXMERT, ViLBERT, UniT, FLAVA, CLIP, VSE++, and VSRN) fails to correctly pair the images and captions, except the large checkpoint of ViLLA by a very thin margin (0.00013 confidence).

▸ Source: Thrush et al., 2022

CLIP and Relatives

BEFORE AND AFTER CLIP

Pre-CLIP:

- *Multimodal Deep Learning* (Ngiam et al., 2011): deep learning for multimodal inputs.
- *Deep Visual-Semantic Alignments* (Karpathy & Fei-Fei, 2015): aligning text with specific image regions.
- *Show and Tell* (Vinyals et al., 2015): CNNs and RNNs combined to turn images into descriptive language.

Post-CLIP:

- **CLIP**: learns from a vast array of internet image-text pairs; zero-shot learning.
- **GroupViT**: segmentation and semantic understanding combined with language.
- **BLIP**: bidirectional learning between vision and language; generating text from images.
- **OWL-ViT**: object-centric representations, understanding objects in images in context with text.

Losses

CONTRASTIVE LEARNING

- **Contrastive learning**: an unsupervised method for representation learning.
- Objective: a representation in which similar items are positioned close together and dissimilar items are clearly separated.
- Example: keep the representations of dogs close to other dogs, but far from cats.



CONTRASTIVE LOSS

- One of the first training objectives for contrastive learning. Input: a pair of samples, similar or dissimilar. Goal: map similar samples close in the embedding space, push dissimilar ones apart.
- For samples x_i, x_j with classes y_i, y_j:

$$L = \mathbb{1}[y_i = y_j] \, \|x_i - x_j\|^2 + \mathbb{1}[y_i \neq y_j] \, \max(0, \epsilon - \|x_i - x_j\|^2)$$

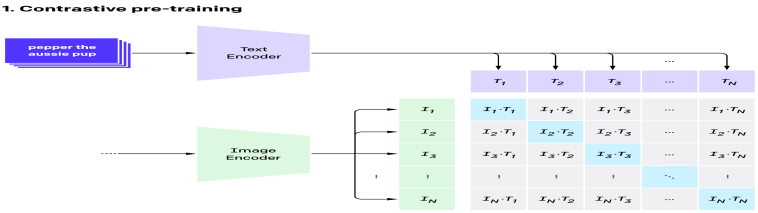
- If the samples are similar ($y_i = y_j$): minimize their squared Euclidean distance — make them closer.
- If they are dissimilar ($y_i \neq y_j$): minimizing $\max(0, \epsilon - \|x_i - x_j\|^2)$ means maximizing their distance, up to a margin ϵ.

Contrastive Language-Image Pre-training (CLIP)

CLIP

- CLIP ▸ Radford et al., 2021 grasps visual concepts through **natural language supervision**: a text encoder and an image encoder are trained jointly on matching captions with their images.
- It adapts to a variety of visual classification benchmarks by simply receiving the *names* of the visual categories to be recognized — "zero-shot" capabilities akin to those of GPT-2 and GPT-3.

CONTRASTIVE PRE-TRAINING



▸ Source: OpenAI

- Given a batch of image-text pairs, compute the cosine similarity matrix between all possible (image, text) candidates.
- Maximize the similarity of correct pairs (blue diagonal), minimize it for incorrect pairs, via a **symmetric cross-entropy loss** over the similarity scores.
- In simple terms: each image should be most similar to its own caption, and each caption to its own image.

CLIP: ENCODERS, USE CASES, LIMITATIONS

- **Independent encoders** for images and text: e.g. a Vision Transformer or a ResNet as image encoder. Switching an encoder requires retraining, as the embedding distribution changes.
- **Use cases**: zero-shot image classification, similarity search, conditioning diffusion models.
- **Limitations**:
  - Unlikely to outperform a specialized, fine-tuned model; generalization is limited for data not encountered during training.
  - Effectiveness and biases depend on the choice of categories: on the FairFace dataset, gender accuracy was over 96%, racial accuracy around 93%.

CLIP: ZERO-SHOT CLASSIFICATION IN PRACTICE

```
from transformers import CLIPProcessor, CLIPModel

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

inputs = processor(text=["a photo of a cat", "a photo of a dog"],
                   images=image, return_tensors="pt", padding=True)
outputs = model(**inputs)
probs = outputs.logits_per_image.softmax(dim=1)
```

- Zero-shot classification of a photo of cats between `dog` and `cat`: the class names become text prompts, and the image is matched against them.
- Result: “a photo of a cat” 99.49%, “a photo of a dog” 0.51%.

Multimodal Text Generation (BLIP)

BLIP: BOOTSTRAPPING LANGUAGE-IMAGE PRE-TRAINING

- BLIP ▸ Li et al., 2022 extends multimodal models to **text generation**.
- **CapFilt (caption and filtering)**: web-scraped image/alt-text pairs are noisy — alt-texts often do not describe the image. One model filters out noisy pairs, another creates captions for images; both are first fine-tuned on a human-annotated dataset. Cleaning the data this way beat training on the raw web data.
- Takeaway: dataset **quality** matters, not just quantity.
- Successor: BLIP-2 ▸ Li et al., 2023, e.g. for visual question answering.

BLIP: ARCHITECTURE

A vision encoder plus a **Multimodal mixture of Encoder-Decoder (MED)**:

- **Vision Transformer**: a plain ViT with a `[CLS]` token for the embedding.
- **Unimodal text encoder**: like BERT, with a `[CLS]` token; trained with a contrastive loss like CLIP to align image and text representations.
- **Image-grounded text encoder**: `[CLS]` replaced by an `[Encode]` token; cross-attention layers combine image and text embeddings into a multimodal representation; a linear layer judges whether image and text match.
- **Image-grounded text decoder**: bidirectional self-attention replaced by **causal** self-attention; trained autoregressively with cross-entropy, for caption generation or visual question answering.

Multipurpose Models

MULTIPURPOSE MODELS

- **Multipurpose models** are multimodal-multitask models.
- They mimic human intelligence by marrying multiple perceptions and transferring knowledge about different tasks for better generalization.
- Main question: how is combining different modalities and tasks achieved?
- Four models: MultiModel (2017), UniT (2021), OFA (2022), Gato (2022).

MULTIMODEL

- A pre-Transformer-era model that combines architectural approaches from different fields. Four modules:
  - **modality nets**: translate between real-world data and a unifying vector space (a tokenizer for language, convolutions for vision; also audio and categorical), and back for the output;
  - **encoder**: receives the input from the modality nets and passes its output to the I/O mixer and the decoder;
  - **I/O mixer** and **decoder**: combine their dual input using cross-attention; the decoder produces the output sequence autoregressively.
- Trained on eight datasets (six language, plus COCO and ImageNet): below state-of-the-art and not significantly better than specialist models — but much better on a low-resource task.

UNIFIED TRANSFORMER (UNIT)

- Built on the Transformer with both encoder and decoder.
- **Modality-specific encoders**: BERT for text, a DETR encoder for vision (a CNN creates a lower-dimensional representation, reorganized as a sequence); more encoders may be added.
- A task-specific token is added to the encoder inputs; the encoder outputs are concatenated into a single sequence.
- The **decoder** follows DETR: it is fed task-specific query vectors instead of the previously produced sequence (not autoregressive); task-specific heads shape the output.
- The general model outperformed its single-task version on multimodal tasks but not on unimodal tasks; easily extendable by using available encoder models.

OFA: SEQUENTIALIZATION IS ALL YOU NEED

- All input is transformed into a **seq2seq problem** and embedded in a shared representation space — no modality-specific modules.
- Images: as in ViT, flattened 16×16 patches, in the same dimension as the text token embeddings.
- Output: a sequence again — images as a sequence of tokens, bounding boxes as a special sequence; another model turns the tokens into images or boxes.
- Autoregressive, cross-entropy loss; trained on cross-modal tasks (visual grounding, grounded captioning, image-text matching, captioning, VQA) and unimodal tasks.
- Outperformed state-of-the-art models on cross-modal tasks such as captioning and VQA; showed transfer to unseen tasks, but was not evaluated against a specialist baseline.

GATO: A GENERALIST DECODER

- One model as a language model, an agent playing games, and an agent controlling robots: a **decoder-only** Transformer on a seq2seq problem.
- Every input is represented sequentially: images as 16×16 patches through a ResNet, text as tokens, game buttons and continuous robot movements tokenized in a vocabulary. A separator token "|" separates observations from the following action:

$$\Big[\ldots \big[x_{\text{Text}}, x_{\text{Images}}, x_{\text{Discrete and Continuous Values}}, |, y_{\text{Action}} \big]_i, \ldots \Big]$$

GATO: TRAINING AND RESULTS

- The next action is predicted autoregressively; a mask in the cross-entropy loss restricts the loss to the action (for text, every token).
- Worse than specialist agents on reinforcement-learning tasks; on unseen tasks it needed fine-tuning, and the results were mixed: sometimes massive fine-tuning led only to small gains, but most of the time it outperformed the specialist agent.

MULTIPURPOSE MODELS: COMPARISON

Model	Approach	Modalities	Outperformed specialist?	Year
OFA	Seq2Seq	Vision, Text	—	2022
Gato	Seq2Seq	Vision, Text, Robotics, Discrete	In most cases	2022
UniT	m encoders, task-specific head	Vision, Text	No	2021
MultiModel	Modality nets for Seq2Seq	Vision, Text, Audio, Categorical	Comparable	2017

- A trend toward seq2seq models: the Transformer used entirely (OFA) or decoder-only (Gato), with a shared representation space and a particular ordering of input and output.
- UniT is more pragmatic: m modalities encoded as a sequence, fused by the Transformer decoder.
- None besides OFA achieved state-of-the-art results; compared to specialists, the general models were comparable.

Introduction to Video

WHAT IS A VIDEO?

- An image is a two-dimensional representation of visual data; a **video** displays such frames sequentially — played at a conventional speed, they give the illusion of motion (like a flipbook).
- Aspects of a video:
  - **Resolution**: the number of pixels in each frame, e.g. HD (1280×720), Full HD (1920×1080), 4K (3840×2160). More detail, but more storage and processing power.
  - **Frame rate**: frames displayed per second, commonly 24, 30, 60 fps. Higher frame rates give smoother motion.
  - **Bitrate**: the quantity of data needed to describe audio and video; higher bitrate, better quality, more storage and bandwidth.
  - **Codecs** (compressor-decompressor): lossless or lossy compression of digital media.

VIDEO PROCESSING

- Video processing automatically analyzes video data to understand and interpret both **temporal and spatial** features: video is a sequence of time-varying images, digitized both spatially and temporally.
- Traditionally, deep learning research focused on images, speech, and text; video offers a valuable opportunity due to its size and complexity.
- Applications:
  - **surveillance**: public safety, crime prevention, traffic monitoring;
  - **autonomous driving**: navigation, obstacle detection, recognizing road signs;
  - **healthcare**: diagnostics, feedback during surgery, patient monitoring.

CHALLENGES IN VIDEO PROCESSING

- **Computational demands**: real-time video analysis requires substantial processing power.
- **Storage requirements**: high-resolution videos generate large volumes of data; efficient compression and data management are necessary.
- **Privacy and ethical concerns**: especially in surveillance and healthcare, video processing handles sensitive information.

Transformers in Video Processing

VIT: PERFORMANCE AND LIMITATION

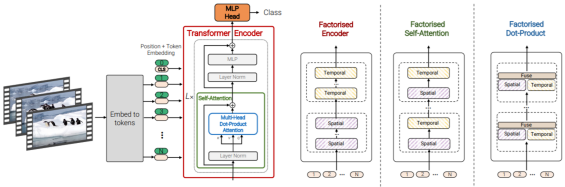
	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

▸ Source: Dosovitskiy et al., 2020

- ViT outperformed other state-of-the-art models, but training it required 2,500 TPU-v3 days. At roughly \$2 per TPU-v3 core-hour, one training run costs about $2 \times 24 \times 2500 =$ \$120,000.

VIDEO VISION TRANSFORMER (VIVIT)

- ViViT ▸ Arnab et al., 2021 extends ViT to video classification. Key issues: train more quickly and efficiently, and handle **temporal** as well as spatial information.
- Extracts spatio-temporal tokens from the video and encodes them with Transformer layers; efficient variants factorise the spatial and temporal dimensions; pre-trained image models make training on comparatively small datasets possible.

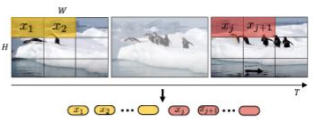


▸ Source: Arnab et al., 2021

EMBEDDING VIDEO CLIPS

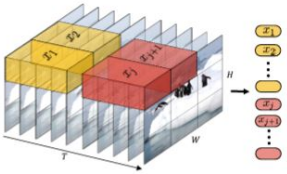
- Embeddings: dense vectors capturing meaningful features. For video, they must preserve **spatial** (what is in each frame) and **temporal** (how things change) information, while reducing dimensionality so processing stays feasible.

Uniform frame sampling



Sample frames uniformly over time, e.g. one in every 2 frames.

Tubelet embedding



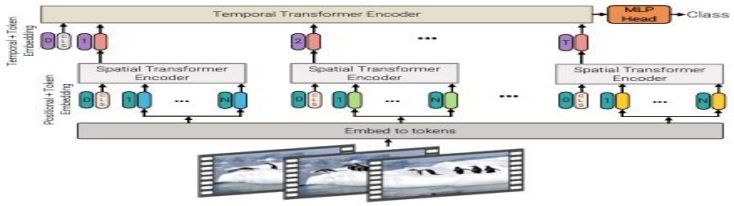
Extract spatio-temporal "tubes" and project them linearly: fuses space and time already during tokenization.

Figures: ▸ Arnab et al., 2021

VIVIT MODEL 1: SPATIO-TEMPORAL ATTENTION

- The natural extension of ViT: each of the n_t frames is split into $n_w \times n_h$ patches, giving $n_t \times n_w \times n_h$ **spatio-temporal tokens**.
- All tokens go through the Transformer encoder together, so each token is contextualized with all others across space and time — patterns in position, color and movement.
- Complexity: $\mathcal{O}(n_h^2\, n_w^2\, n_t^2)$.
- Attention over all spatio-temporal tokens is expensive; uniform frame sampling and tubelet embedding help reduce the cost.

VIVIT MODEL 2: FACTORISED ENCODER

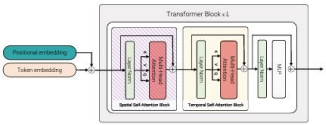


▸ Source: Arnab et al., 2021

- Model 1 contextualizes all patches simultaneously — inefficient. Model 2 separates spatial and temporal encoders, sequentially:
  - a **spatial** Transformer encoder (= ViT) contextualizes within each frame and encodes it into a single embedding;
  - a **temporal** Transformer encoder then processes the frame embeddings.
- Complexity: $\mathcal{O}(n_h^2\, n_w^2 + n_t^2)$.

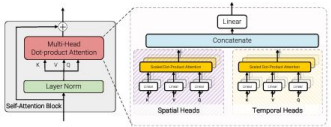
VIVIT MODELS 3 AND 4

Model 3: Factorised Self-Attention



First self-attention **spatially** (among tokens of the same time index), then **temporally** (among tokens of the same spatial index). No `[CLS]` token, because of ambiguities.

Model 4: Factorised Dot-Product Attention



Half of the attention heads use keys and values from the same spatial indices, the other half from the same temporal indices.

Complexity of both: same as Model 2. Figures: ▸ Arnab et al., 2021

ViViT: Experiments and Discussion

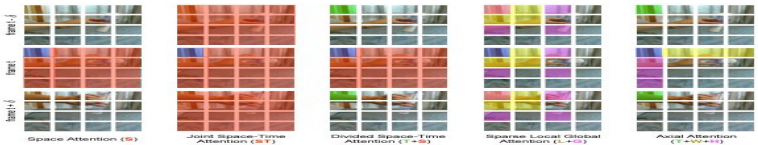
	K400	EK	FLOPs ($\times 10^9$)	Params ($\times 10^6$)	Runtime (ms)
Model 1: Spatio-temporal	80.0	43.1	455.2	88.9	58.9
Model 2: Fact. encoder	78.8	43.7	284.4	115.1	17.4
Model 3: Fact. self-attention	77.4	39.1	372.3	117.3	31.7
Model 4: Fact. dot product	76.3	39.5	277.1	88.9	22.9
Model 2: Ave. pool baseline	75.8	38.8	283.9	86.7	17.3

▸ Source: Arnab et al., 2021

- Model 1 achieves the best accuracy but needs the longest training; Model 2 gives relatively high accuracy with shorter training than Models 3 and 4 — the most efficient overall.
- Like ViT, ViViT needs extremely large datasets, which rarely exist for video. Approach: first pre-train on a large image dataset with ViT to initialize the model.

TIMESFORMER

Concurrent work with ViViT ▸ Bertasius et al., 2021; five space-time self-attention schemes (blue: query patch):



- **Space**: as in ViT, within one frame. **Joint space-time**: across frames, as ViViT Model 1.
- **Divided space-time**: first temporally (same position, other frames), then spatially — similar to ViViT Model 3.
- **Sparse local global**: selectively combines local and global information. **Axial**: spatial and temporal dimensions separately along their axes.
- **Divided space-time attention** gives the best balance of parameter efficiency and accuracy (K400, SSv2).

CONCLUSION: VIDEO TRANSFORMERS

- ViViT extended ViT to video with several models (factorised encoder, factorised self-attention, factorised dot-product attention), all aimed at managing the space-time dimensions efficiently.
- TimeSformer likewise evolved from ViT, using diverse attention mechanisms over space and time.
- Key takeaway: the focus is on reducing the significant computational cost of applying Transformers to video, enabling learning with fewer computational resources.

License & Attribution

LICENSE & ATTRIBUTION

- **Main source.** All sections except those listed below follow the **Hugging Face Community Computer Vision Course** (`https://huggingface.co/learn/computer-vision-course`, repository `https://github.com/johko/computer-vision-course`, © 2023 Johannes Kolbe and contributors) — licensed **MIT**. Units 1 (*Introduction*), 3 (*Vision Transformers*), 4 (*Multimodal Models*) and 7 (*Video*). The course is prose, not slides: frames condense its text into bullet points without adding content.
- **Supplement.** "Transformers in Computer Vision", "Multimodal Architectures" and "Multipurpose Models" condense text from the LMU booklet ***Multimodal Deep Learning*** (SLDS, LMU Munich, `https://slds-lmu.github.io/seminar_multimodal_dl/`, chapters 2.2, 3 and 4.3) — licensed **CC BY-NC-SA 4.0** (`https://creativecommons.org/licenses/by-nc-sa/4.0/`): non-commercial use, with attribution; these three sections are adapted from it and are therefore shared under the same license. The ViT figure in that section is from the booklet, which takes it from the paper.
- **Figures** are those the course uses, with the attribution the course gives: most are taken from the original papers (linked on each frame); the CLIP pre-training figure is from OpenAI; the MaskFormer figure comes via the Hugging Face documentation.